
MEMAUDIT: An Exact-Oracle Evaluation Protocol for Budgeted Long-Term LLM Memory Writing

Anonymous Author(s)

Affiliation

Address

email

Abstract

Long-term LLM agents must decide what to write into persistent memory before future queries are known, but existing evaluations conflate memory writing, retrieval, and reader reasoning. We introduce MEMAUDIT, a reusable exact-oracle evaluation protocol for budgeted memory writing. A package contains an experience stream, candidate memory representations, storage costs, semantic evidence units, and future-query requirements; together these turn long-term memory writing into an auditable finite optimization problem with a certified denominator. We instantiate a standard concave-over-modular semantic coverage objective under a storage budget and a one-representation-per-experience constraint, then compute package optima with branch-and-bound/MILP. Across controlled, validity-heavy, human-validated natural support-sliced, and exported-system packages, MEMAUDIT exposes representation, validity, and budget-selection effects that end-to-end QA alone cannot localize. Exported Mem0, A-Mem, and Letta stores show that heterogeneous memory systems can be scored without forcing a shared internal representation. MEMAUDIT provides a reusable evaluation layer for measuring what memory writers preserve under a fixed budget.

1 Introduction

LLM agents increasingly operate across sessions: they converse with users, call tools, edit code, inspect documents, and later need to reuse what happened. A fixed context window makes the naive policy of retaining everything impossible. The agent must compile an experience stream into persistent memory, choosing not only whether to store an item but also which representation should survive: raw span, extracted fact, temporal event, graph edge, summary, rule, skill, compound update, or tombstone.

This paper argues that long-term memory writing should be evaluated as finite semantic compression, not only as an architecture choice. The load-bearing object is an oracle package: for a fixed budget, finite candidate memories, and fixed future-query requirements, how close is a writer to the best package-feasible store? This separates write quality from downstream retrieval and reader behavior. A system may fail because it wrote the wrong memory, because retrieval missed a good memory, or because the reader ignored evidence; final QA accuracy alone cannot identify which layer failed.

The distinction matters because most deployed memory systems are evaluated only after several other choices have intervened. A memory writer may extract a useful fact but store it in a form too expensive to keep under the target budget. A retriever may miss the relevant memory even if the writer preserved it. A reader may see the evidence and still answer incorrectly. These are different engineering failures. MEMAUDIT isolates the first one by freezing a finite candidate set, a storage

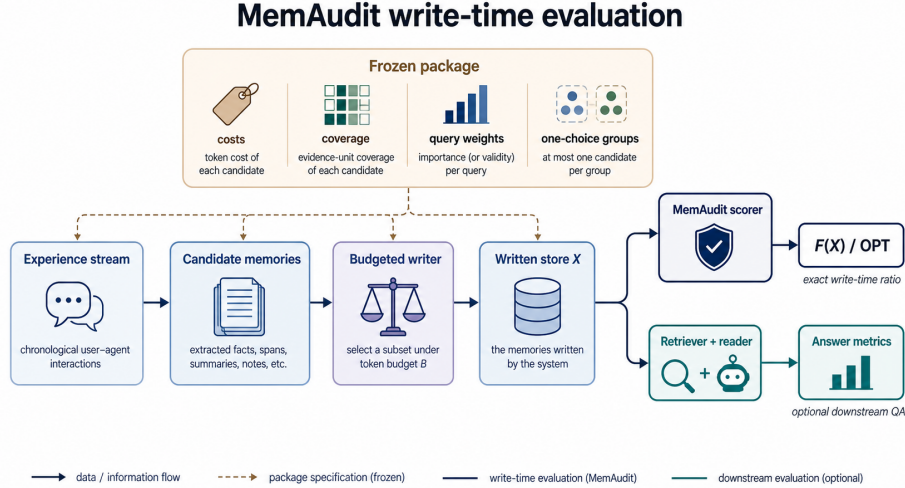


Figure 1: MEMAUDIT evaluates the written memory store before retrieval and reader reasoning. The frozen package defines the exact write-time denominator, while downstream QA is measured separately through the optional retriever/reader branch.

accounting rule, and an evidence-unit objective. The resulting denominator is not a claim about all possible memories; it is a reproducible answer to the narrower question “given this package, how much of the package-feasible semantic value did the writer preserve?”

Contributions. We make four evaluation-and-dataset contributions. (i) We define a finite MEMAUDIT package for budgeted memory writing with a package oracle ratio and a union denominator for external stores. (ii) We instantiate the known concave-over-modular submodular coverage structure for semantic memory preservation. (iii) We provide exact/certified package optima and an oracle reference baseline, with an independent MILP audit. (iv) We release an initial MEMAUDIT suite spanning controlled exact packages, validity-heavy stress packages, human-validated model-adjudicated natural support slices with reported inter-annotator agreement, and exported-system packages for Mem0, A-Mem, and Letta.

E&D artifact role. MEMAUDIT is designed to complement systems such as Mem0, Letta, A-Mem, MemGPT-style archival memory, and graph-based memories by scoring their exported writes under a shared denominator. The released artifact contains deterministic exact-package generators, certified solvers, cached natural package exports, adjudication summaries, exported Mem0/A-Mem/Letta memory stores, and scripts to reproduce the main tables and figures without additional API calls. API-backed construction is needed only to regenerate natural annotations or rerun external memory exports.

Relation to MemSim. MemSim/MemDaily [14] automatically constructs reliable QA pairs for evaluating personal-assistant memory through downstream answering. MEMAUDIT targets a complementary layer: it evaluates the write-time memory store itself by defining finite candidate representations, costs, evidence units, and an exact budgeted optimum. Thus MemSim asks whether an agent answers generated memory questions correctly, while MEMAUDIT asks how close a written memory store is to the best package-feasible semantic store before retrieval and reader reasoning are invoked. This also lets MEMAUDIT localize where the problem sits: candidate generation, representation choice, budget-aware selection, retrieval, or reader use.

2 MEMAUDIT Package

An LLM agent observes experiences $\mathcal{E} = (e_1, \dots, e_T)$. Each experience can generate multiple candidate write choices, such as raw text, an atomic fact, a summary, a graph edge, a rule, a

64 compound update, or a tombstone. Let

$$\mathcal{U} = \{(i, j) : i \in [T], j \in J_i \setminus \{\text{discard}\}\}$$

65 be the virtual ground set of non-discard experience-representation choices. Each element $u = (i, j)$
 66 has cost $c_u > 0$, and $G_i = \{(i, j) : j \in J_i \setminus \{\text{discard}\}\}$ is the group of choices for experience i . A
 67 feasible memory store satisfies one storage budget and one choice per experience:

$$\mathcal{F}_B = \left\{ X \subseteq \mathcal{U} : \sum_{u \in X} c_u \leq B, \quad |X \cap G_i| \leq 1 \forall i \right\}. \quad (1)$$

68 Thus the feasibility structure is the intersection of one knapsack constraint and one partition matroid.

69 This virtual-ground-set view makes representation choice explicit instead of treating memory as a
 70 homogeneous list of strings. For example, suppose a user first says they prefer vegetarian travel
 71 meals and later says they are pescatarian now. A future query asks what meal should be booked.
 72 The package may contain a stale vegetarian fact, a current pescatarian fact, a raw span containing
 73 both turns, a tombstone invalidating the old preference, and a compound update saying “vegetarian
 74 is superseded by pescatarian.” These candidates have different costs and cover different evidence
 75 units: current truth, invalidation, temporal order, or raw provenance. The constraint $|X \cap G_i| \leq 1$
 76 prevents the package optimum from keeping every representation of one experience and makes
 77 the denominator comparable to a writer that chooses one persistent form. Because some deployed
 78 systems keep raw+summary+fact variants, the artifact also supports a relaxed $|X \cap G_i| \leq k$ audit;
 79 on Natural-87, allowing $k = 2$ changes exact OPT by only 1.3–6.0% across the evaluated budgets
 80 (Section C).

81 The package also fixes storage accounting. In our experiments, the default cost is normalized
 82 word-equivalent storage over the serialized memory exposed to the writer/reader. This is a declared
 83 benchmark cost, not a universal systems-cost model: real systems also pay embedding-index slots,
 84 graph-node overhead, JSON metadata, and retrieval compute. To check that the main diagnostics
 85 are not an artifact of whitespace word counts, Section C recomputes the Natural-87 package and
 86 exported-store pruning under an alternative cost $c(u) = 8 + \lceil \text{bytes}(u)/24 \rceil$, approximating fixed
 87 per-record overhead plus serialized payload. The qualitative exported-system ordering is stable under
 88 this rule. When external systems export memories outside the package candidate set, those memories
 89 are added to a union package with their own measured costs rather than forced into the original
 90 candidate taxonomy.

91 The benchmark defines semantic evidence units $\mathcal{R} = \{r_1, \dots, r_M\}$. A future query q has required
 92 evidence units $R(q) \subseteq \mathcal{R}$ and nonnegative importance weights, inducing evidence weights $w_r \geq 0$.
 93 Each candidate memory has a nonnegative coverage row $a_{ur} \in [0, 1]$. The utility of a store is

$$F(X) = \sum_{r \in \mathcal{R}} w_r h_r \left(\sum_{u \in X} a_{ur} \right), \quad (2)$$

94 where each h_r is concave, nondecreasing, and normalized. The default package objective uses
 95 $h_r(z) = \min(1, z)$, so duplicate memories of the same evidence unit have diminishing marginal
 96 value.

97 Evidence units are the semantic atoms used by the benchmark. In a synthetic package they are
 98 generated from the hidden event graph; in a natural support slice they are source-backed units
 99 extracted from the support sessions and then mapped to future-query requirements. A unit can
 100 represent a fact, a temporal relation, an entity preference, a deletion, an abstention condition, or
 101 a validity-state update. The objective is positive coverage: stale-fact avoidance is represented by
 102 covering the evidence unit that says an older fact is no longer current, not by assigning negative
 103 utility to stale memories. This keeps the objective monotone while still allowing the benchmark to
 104 ask whether writers preserve current-truth information.

105 The concavity in Equation (2) encodes diminishing returns. If two candidates cover the same evidence
 106 unit, the second copy should usually help less than the first. If a query requires multiple evidence
 107 units, the weights w_r distribute value across those requirements. The objective is therefore a semantic
 108 surrogate for write-time preservation, not a replacement for downstream QA. A reader can still fail
 109 with a high- F store, and a low- F store might answer an easy question by chance. The point is that F
 110 gives a deterministic, package-local target for the writer layer.

111 **Theorem 1** (Semantic coverage is monotone submodular). *Let $w_r \geq 0$, $a_{ur} \geq 0$, and let each h_r be*
 112 *concave, nondecreasing, and satisfy $h_r(0) = 0$. Then F in Equation (2) is normalized, monotone*
 113 *nondecreasing, and submodular on $2^{\mathcal{U}}$.*

114 The proof is the standard concave-over-modular diminishing-returns argument and is given in
 115 Section A. The theorem supports the semantic surrogate; it does not claim black-box LLM answer
 116 accuracy is submodular.

117 **Definition 2** (MEMAUDIT package and package ratio). *An MEMAUDIT package is a tuple*

$$\mathcal{P} = (\mathcal{U}, \mathcal{G}, c, \mathcal{R}, A, w, B).$$

118 *Its exact package optimum and package ratio are*

$$\text{OPT}_{\mathcal{P}}(B) = \max_{X \in \mathcal{F}_B(\mathcal{P})} F_{\mathcal{P}}(X), \quad \rho_{\mathcal{P}}(X) = F_{\mathcal{P}}(X)/\text{OPT}_{\mathcal{P}}(B).$$

119 **Definition 3** (Union denominator for external stores). *If an external memory system writes memories*
 120 *Y not contained in $\mathcal{U}$, we evaluate them in the finite union package $\mathcal{P}^+(Y)$ obtained by adding Y ,*
 121 *their costs, and their adjudicated coverage rows. The external-store ratio is*

$$\rho_{\text{union}}(Y) = \frac{F_{\mathcal{P}^+(Y)}(Y)}{\text{OPT}_{\mathcal{P}^+(Y)}(B)}.$$

122 *We also report an analysis-only upper-pruned bound over subsets of Y to separate extraction quality*
 123 *from budget-aware selection.*

124 The union denominator is essential for scoring real systems. Suppose Mem0 writes a memory that is
 125 not one of our package candidates. Scoring it only against the package candidate optimum would be
 126 ambiguous: the system may have created a useful representation that the package did not contain. In
 127 $\mathcal{P}^+(Y)$, the exported memory becomes a first-class candidate with an adjudicated coverage row. The
 128 numerator scores exactly what the system wrote and retained, while the denominator asks what the
 129 best budget-feasible subset could have achieved using both package candidates and system exports.
 130 This makes low scores interpretable: a low raw exported-store ratio with a high upper-pruned bound
 131 indicates that the system extracted useful content but did not select or compact it well under budget.

132 3 Exact Optima and Reference Writers

133 Exact optimization is central to MEMAUDIT. For small packages, we compute $\text{OPT}_{\mathcal{P}}(B)$ by
 134 branch-and-bound over experience-representation assignments. For the default clipped-coverage
 135 objective, we also use a MILP certificate with binary candidate variables x_u and coverage variables
 136 $y_r \leq \sum_u a_{ur} x_u$, $y_r \leq 1$. Greedy or learned references are never labeled as OPT.

137 The branch-and-bound solver searches over groups rather than over unconstrained candidate subsets.
 138 At each group it branches over discard or one representation choice, tracks remaining budget, and
 139 uses an optimistic fractional bound over current marginal gains to prune subtrees. The bound is
 140 admissible for the clipped coverage objective because future marginal coverage can only decrease as
 141 more evidence units are covered. This solver is sufficient for the exact-small and adjudicated natural
 142 packages we report. The MILP audit is included to reduce the risk that an implementation detail in
 143 the custom solver defines the benchmark.

Table 1: Exact-solver certification. PuLP MILP and pure-Python branch-and-bound were run on the same requested-scope audit instances; equality is objective-value equality, allowing tied optimal stores with different candidate ids.

Audit	Rows	Objective matches	Max diff
B&B vs MILP	1,200	1,200/1,200	0.0

144 The oracle reference baseline is grouped value-threshold (GVT). For each arriving experience, GVT
 145 forms the budget-feasible candidates whose oracle marginal density exceeds a threshold, then inserts
 146 the admissible representation with largest raw marginal value. A threshold grid gives a conservative
 147 insertion-only constant-factor guarantee under exact marginals and small-item assumptions; the

theorem and proof are in [Section A](#). GVT is included as a calibration baseline: because it has access to exact package marginals, it helps verify that the package and solver behave sensibly. It is not the proposed deployed writer.

Density-only representation choice is included as a negative control. It can be arbitrarily bad even for modular utility: a tiny candidate can have higher density while losing nearly all value. Controlled experiments below show this failure empirically.

This calibration role is useful because it makes representation-choice effects visible. A pure density rule may prefer a cheap but narrow memory; a pure value rule may spend the budget too quickly on expensive raw spans. GVT combines a density threshold with within-group value choice, which is well aligned with the one-representation-per-experience structure, while remaining secondary to the benchmark artifact itself.

Proposition 4 (Density-only can be arbitrarily bad). *For every $\eta > 0$, there is a one-experience modular instance satisfying $0 < c_u \leq B/2$ in which a max-density representation rule obtains at most η times the finite package optimum.*

Theorem 5 (Scoped oracle-GVT guarantee). *Let F be normalized, monotone, and submodular on $\mathcal{U}$. Suppose candidates arrive group-by-group, the algorithm is insertion-only, computes exact oracle marginals, enforces [Equation \(1\)](#), and chooses the maximum raw marginal among threshold-admissible candidates in each arriving group. If every non-discard representation has cost $0 < c_u \leq B/2$ and the threshold grid contains*

$$\lambda \in \left[(1 - \varepsilon) \frac{\text{OPT}}{2B}, \frac{\text{OPT}}{2B} \right],$$

then the best grid solution satisfies $F(X) \geq (1 - \varepsilon)\text{OPT}/4$. This is a calibration result for the oracle baseline; deployed writers without exact marginals are evaluated empirically.

4 Controlled Exact-OPT Packages

MEMAUDIT-Small generates compact hidden event graphs with raw, fact, summary, tombstone, and compound-update candidates. The exact 500-seed budget sweep maps fractional budgets 0.01, 0.02, 0.05, 0.10, 0.20 to absolute budgets $B = 1, 2, 4, 8, 16$; we discuss the non-degenerate budgets $B = 2, 4, 8, 16$.

Each package instance contains enough redundancy to make representation choice nontrivial. Raw spans are broad but expensive; atomic facts are compact but may miss temporal or validity relations; summaries can cover several units at intermediate cost; tombstones and compound updates encode current-state transitions that are not recoverable from a stale fact alone. Baselines are constrained to the same package and budget. Recency raw keeps recent raw memories until the budget is exhausted. Fact-only and summary-only restrict the candidate set to one representation family. Density-only chooses the candidate with highest current marginal value per unit cost. No-tombstone OPT removes validity candidates and then resolves the exact problem, measuring the frontier loss from omitting that representation class.

[Figure 2](#) shows that the exact package denominator separates memory-writing policies. MEMAUDIT-GVT covers all invalidation units selected by OPT at the reported budgets (mean invalidation coverage 2.00 for each of $B = 2, 4, 8, 16$). Density-only covers no invalidation units at $B = 2, 4, 8$ and reaches only 0.264–0.361 of OPT there. Tombstone/compound-update candidates expand the feasible frontier: exact no-tombstone OPT reaches 0.523, 0.654, 0.689, 0.815 of full OPT across $B = 2, 4, 8, 16$.

The exact sweep is the cleanest evidence that MEMAUDIT measures the intended layer. The denominator is certified, all methods face the same candidate set and budget, and the gaps are not downstream retrieval artifacts. The strongest negative result is not that one heuristic loses on one seed, but that a plausible local rule systematically discards the validity-state units that exact OPT preserves. The no-tombstone OPT curve is also important because it is not an algorithm comparison: even the best package-feasible store is worse when the candidate generator lacks current-state representations. This distinguishes candidate-generation bottlenecks from selection bottlenecks.

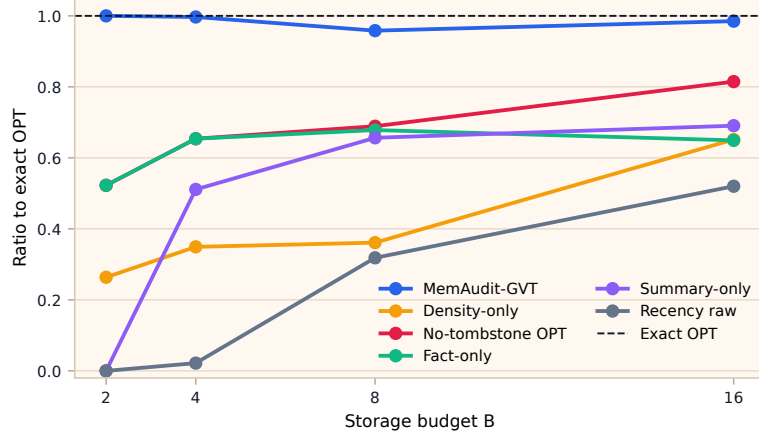


Figure 2: Exact-small budget sweep. MEMAUDIT-GVT remains near certified package OPT, while density-only, recency raw, and single-representation baselines degrade under tight storage. Bands are bootstrap 95% intervals from the canonical 500-seed run.

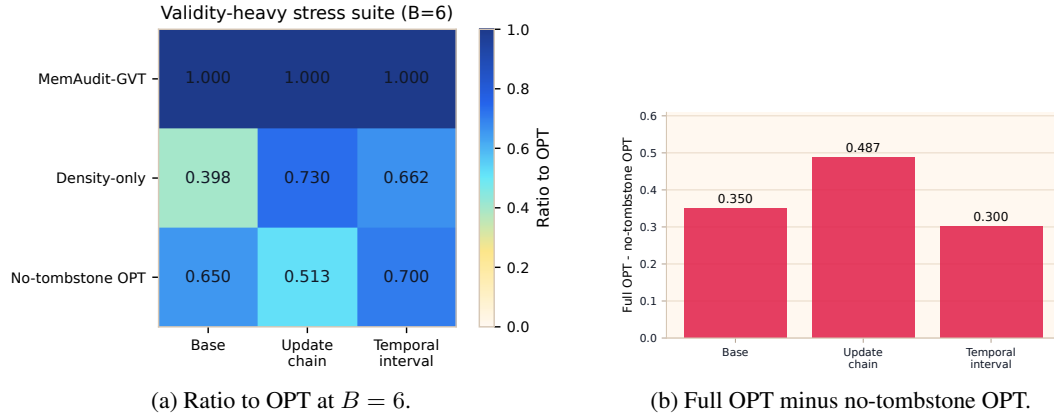


Figure 3: Validity-heavy stress visualization for the headline exact distributions: base, update-chain, and temporal-interval.

5 Validity-State Stress Packages

Memory systems often fail by preserving stale facts. We encode current truth, supersession, deletion, and abstention as positive validity-state evidence units rather than negative utility. A tombstone therefore covers an invalidation unit such as “the old address is no longer current,” while stale retrieval and stale answers remain separate diagnostic metrics.

This design choice is what lets validity fit into a monotone objective. A memory store is rewarded for preserving the evidence needed to answer “what is true now?” or “should the agent abstain because the previous answer is invalid?” It is not directly penalized for retaining old memories. A stale raw span may still be useful if paired with a later tombstone or compound update, because the reader can infer the update sequence. Without such validity-state candidates, a writer can spend the budget on high-density facts and still fail on future queries that ask for current truth.

The stress suite sharpens the validity-memory claim. At $B = 6$, no-tombstone OPT falls to 0.513 on `update_chain` and 0.700 on `temporal_interval`, so validity-state candidates expand the package-feasible frontier in controlled update-heavy instances. Other stressors are useful diagnostics, but the main paper restricts claims to the distributions in Figure 3.

The three headline distributions cover different validity pressures. The base distribution has ordinary preferences and events with occasional updates. The `update_chain` distribution creates repeated supersession, where the useful memory is often the latest current-state representation plus evidence

that older states are obsolete. The `temporal_interval` distribution stresses interval boundaries and recency of truth, so compact temporal updates matter more than raw recency alone. The full stress suite also includes density traps, scope shifts, redundancy-heavy packages, and hard abstention cases, but those are deferred because the central main-text claim is narrower: validity-state representation choices can expand the package-feasible semantic frontier.

6 Natural and Exported-System Packages

Natural packages demonstrate that the same finite-package schema can be populated from realistic conversation support slices. We built a 200-example LongMemEval-style support-slice package with Gemini Flash-Lite through OpenRouter. Candidate generation sees answer-support sessions; query and gold-answer fields are used only in the separate benchmark-labeling pass that maps evidence units to query requirements. The package is support-sliced, not full-haystack LongMemEval, and the labels are source-backed model-adjudicated labels with explicit rejection and audit paths.

The natural construction route is intentionally auditable. The pipeline separates three artifacts: candidate memories, evidence units, and query requirements. Candidate memories are generated from support sessions. Evidence units are grounded in the same source text. Query requirements are assigned in a separate labeling pass. This separation reduces direct answer leakage into the writer while still allowing exact package scoring after the artifact is frozen.

Table 2: Natural package construction reliability. The package format exposes which examples are schema-valid, adjudicated for scoring, or flagged for ambiguity instead of silently mixing them into one benchmark pool.

Artifact	Scope	Resolved/agreement	Disputed	Role
Natural-200 primary	200 ex.	200 schema-valid	–	scalable package construction
Adjudicated subset	100 ex.	87 accepted/corrected	13	main natural scoring set
Flash-Lite spot-check	30 ex.	29 accepted/corrected	1	independent consistency check
Human-edited seed set	100 ex.	100 schema-valid	–	prompt/schema development

Table 2 is intentionally conservative. The primary Natural-200 export used 681 API calls, 1,978,327 tokens, and about \$0.907. An initial secondary audit surfaced ambiguous required-unit cases, motivating a stricter Gemini-Flash adjudication pass on 100 examples; it accepted or corrected 87 examples and flagged 13 ambiguous examples. The independent 30-example spot-check accepted/corrected 29 examples. This is an advantage of the package format: ambiguous evidence requirements become measurable dataset objects rather than hidden benchmark noise.

The coverage audit in Table 3 is part of the artifact, not a side note. On the 87-example adjudicated subset, two human annotators labeled all nonzero coverage cells plus a stratified 20% sample of model-zero cells. Human agreement is substantial, Gemini agrees closely with the human-consensus subset, and re-scoring package writers with human labels changes ratios by only 0.035 on average while preserving rankings. All natural-package rows have bootstrap intervals in the artifact; for example, Estimated-GVT at $B = 100$ is 0.829 [0.761, 0.887] under Gemini labels and 0.828 [0.755, 0.893] under human-consensus labels. The Sonnet rows are useful as a stress test: rankings remain stable, but binary labels are not interchangeable with the human-validated Gemini labels.

The next question is whether the same package machinery can score memory systems that write in very different forms. The union denominator makes this possible: exported memories are added as candidates with measured costs and adjudicated coverage rows, so the score can distinguish extraction quality from budget-aware selection. We therefore evaluate Mem0, Letta, and A-Mem exports on the same 87-example adjudicated subset before discussing the specific failure modes.

Figure 4 shows that MEMAUDIT can diagnose heterogeneous memory systems without requiring a common internal representation. The non-oracle Estimated-GVT line is the more realistic package writer: it uses estimated values rather than hidden oracle marginals and reaches 0.53, 0.68, 0.83 of package OPT across $B = 30, 60, 100$. Letta’s core+archival exports achieve the strongest salience-pruned union ratios on the adjudicated subset, indicating that passage-style archival memories can be effective under this budgeted evidence objective. Mem0’s high upper-pruned bound shows that useful evidence is often extracted, while the gap to recency/salience pruning isolates budget-aware selection as the bottleneck. A-Mem illustrates a complementary behavior: rich notes preserve evidence, but

Table 3: Coverage-label audit summary on the 87-example adjudicated subset. Cell-level rows use the released human audit frame; package-pass rows compare model package adjudications on the same cells. Ratio MAD is the mean absolute ratio change at $B = 30, 60, 100$ where a package-level re-score is defined.

Audit pair	Scope	Cohen’s κ	Raw agree.	Ratio MAD	Ranking
Human-human	1,975 cells	0.831	0.918	–	–
Human-Gemini	1,813 agreed cells	0.863	0.934	0.035	preserved
Sonnet-Gemini	package pass	0.649	0.845	0.079/0.073 0.070	preserved, $\rho=1.0$
Sonnet-human	package pass	0.588	0.811	–	preserved
Sonnet-Gemini	blind cell pass	0.531	0.763	–	–
Sonnet-human	blind cell pass	0.484	0.739	–	–

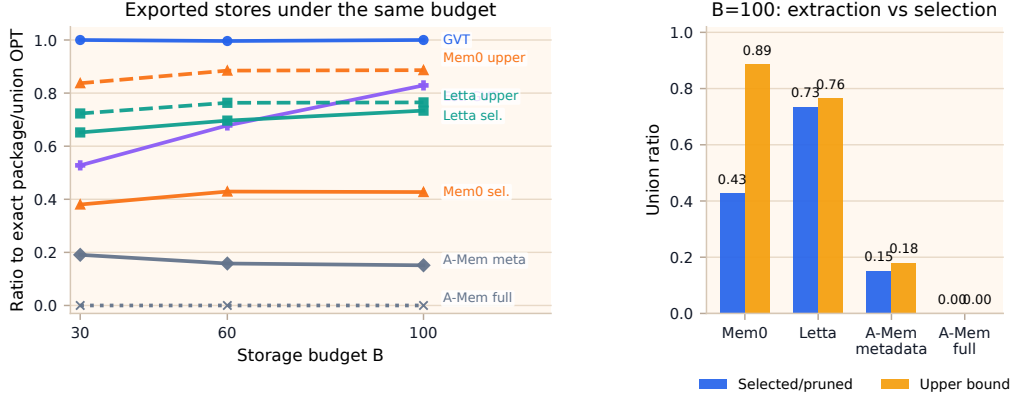


Figure 4: Actual-system diagnostic on the 87-example adjudicated subset. Left: package or union ratio across budgets for oracle MEMAUDIT-GVT, non-oracle Estimated-GVT, and exported Mem0/Letta/A-Mem stores. Right: at $B = 100$, upper-pruned rows isolate extraction potential, while selected/pruned rows show the budget-aware selection actually achieved by each exported store.

258 their serialized size makes them poorly matched to strict small-budget stores unless paired with
259 compaction or admission control.

260 This experiment also illustrates why the union denominator matters. If we scored exported memories
261 only against the original package candidates, a system-specific representation could be unfairly
262 undervalued. If we scored only the system’s own memories without an optimum, we could not
263 tell whether the problem was extraction or budget-aware selection. The union package makes both
264 statements precise: the numerator is the system’s selected store, and the denominator is the best finite
265 package-feasible store after the system’s own memories have been added as candidates. The exactly
266 zero A-Mem full-store row is a cost-model diagnostic rather than a coverage bug: at $B = 100$, no
267 full serialized A-Mem note fits the budget (mean minimum full-note cost 1768 words, or 464 units
268 under the byte-overhead rule), while its compact metadata view remains feasible.

269 The diagnostic also avoids a common evaluation shortcut. It would be easy to ask each memory
270 system to answer downstream questions and report the final score. That number is useful, but it does
271 not say whether a low score came from writing, retrieval, prompt formatting, model capability, or
272 answer normalization. In Figure 4, the same evidence objective, costs, and budgets are used for all
273 exported stores. This makes the failure mode more actionable: improve extraction if the upper-pruned
274 bound is low; improve admission or compaction if the bound is high but the selected-store ratio is
275 low; improve retrieval/reader behavior only after the written store is known to contain the necessary
276 evidence.

277 7 Related Work

278 **Memory systems and benchmarks.** MemGPT, Reflexion, MemoryBank, Mem0, and A-Mem
279 propose memory architectures or policies for long-term agents [9, 10, 15, 3, 13]. LoCoMo and

LongMemEval evaluate long-horizon memory behavior [7, 12]. MemSim/MemDaily automatically constructs reliable personal-memory QA data through Bayesian simulation [14]. MEMAUDIT is complementary to these benchmarks: it supplies a finite write-time denominator before retrieval and reader errors are mixed in.

These systems make different architectural commitments: virtual context management, reflection, personalized memory banks, dynamic memory extraction, graph organization, and admission control. MEMAUDIT does not require them to share an internal representation. Instead, it asks them to export written memories and then scores those memories under a common package or union denominator. This is why the method is positioned as an evaluation protocol rather than another memory architecture.

Retrieval and submodular selection. RAG, MMR, and submodular context selection choose evidence after a query is known [6, 2, 1, 5]. MEMAUDIT instead evaluates persistent write-time memory before future queries are known. Concave-over-modular objectives and knapsack submodular optimization are standard tools [8, 11, 4]; we use them to define and certify a benchmarkable semantic surrogate.

Dataset and evaluation protocols. Several long-memory datasets emphasize whether an agent can answer questions after long interaction histories. MEMAUDIT targets a different but compatible layer: whether the persistent store contains the evidence needed before the query-time pipeline runs. This is closest in spirit to benchmark designs that expose intermediate artifacts rather than only final predictions. The package artifact is intentionally finite and inspectable. Every ratio should be reproducible from a candidate table, a cost vector, a coverage matrix, evidence weights, and a budget. This design makes the benchmark easier to audit than an end-to-end agent transcript where hidden retrieval or prompt-state changes may affect the outcome.

8 Scope, Limitations, and Artifact

MEMAUDIT’s scores are package-conditional by design: the finite candidate table, cost model, evidence schema, and budget define the evaluated object. This does not replace end-to-end assistant evaluation, but it supplies the missing write-time diagnostic. If a generator omits the right memory, the package optimum measures the best store within that table; if a writer extracts useful memories but fails to retain the right subset, union and upper-pruned scores expose the selection bottleneck. The cost model and one-representation constraint are explicit package parameters; the released sensitivity audit recomputes Natural-87 under a byte-overhead cost and a $k = 2$ representation relaxation. The $k = 1$ default is a benchmark choice, not a claim that deployed systems must store one representation per experience; allowing $k = 2$ changes exact OPT by only 1.3–6.0% on Natural-87, so the constraint affects ratios at the percentage-points level rather than changing the qualitative rankings.

The natural support-sliced package is a conservative first natural instantiation. Model-adjudicated labels provide a scalable construction method with explicit rejection, correction, and audit hooks, and the released human coverage audit shows that the resulting coverage rows are stable under independent human review on the adjudicated subset. Support slices isolate the write-time problem before full-haystack expansion introduces retrieval and session-selection confounds.

The oracle reference writer uses exact coverage marginals, so deployed systems will need estimated utilities, learned candidate generators, model-based annotators, compaction, or native budget-aware admission. MEMAUDIT makes those extensions measurable by reporting package ratios, union ratios, and downstream QA metrics side by side. Exact optimization currently targets small and medium packages; larger packages can use decomposition, MILP certificates with gaps, or exact evaluation on sampled slices. The LongMemEval-S transfer study is therefore included as a retrieval/reader diagnostic, not as the primary denominator claim.

The artifact contains exact and stress generators, branch-and-bound and MILP solvers, natural exports, adjudication summaries, public-system scoring scripts, cached API metadata, and figure-generation scripts. Minimal reproduction does not require API calls; API-backed construction is needed only to regenerate natural annotations or rerun public-system exports. We recommend reporting the package schema, cost rule, solver status, oracle ratio, union ratio, and downstream QA metric as separate columns, so write quality, extraction, selection, retrieval, and reader behavior remain auditable.

References

- [1] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, and Andreas Krause. Streaming submodular maximization: Massive data summarization on the fly. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 671–680. ACM, 2014.
- [2] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336. ACM, 1998.
- [3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [4] Chien-Chung Huang, Naonori Kakimura, and Yuichi Yoshida. Streaming algorithms for maximizing monotone submodular functions under a knapsack constraint. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2017)*, volume 81 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:14, Dagstuhl, Germany, 2017. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [5] Lilly Kumari, Shengjie Wang, Tianyi Zhou, Nikhil Sarda, Anthony Rowe, and Jeff Bilmes. BumbleBee: Dynamic KV-cache streaming submodular summarization for infinite-context transformers. In *Proceedings of the First Conference on Language Modeling*, 2024.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [7] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [8] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.
- [9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [11] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.
- [12] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [13] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [14] Zeyu Zhang, Quanyu Dai, Luyu Chen, Zeren Jiang, Rui Li, Jieming Zhu, Xu Chen, Yi Xie, Zhenhua Dong, and Ji-Rong Wen. MemSim: A bayesian simulator for evaluating memory of LLM-based personal assistants. *arXiv preprint arXiv:2409.20163*, 2024.
- [15] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.

380 A Deferred Proofs

381 *Proof of Theorem 1.* Normalization follows from $h_r(0) = 0$. For monotonicity, adding any element
 382 u increases each inner sum by $a_{ur} \geq 0$, and h_r is nondecreasing. For submodularity, let $S \subseteq T \subseteq \mathcal{U}$
 383 and $u \notin T$. Define $z_r(S) = \sum_{v \in S} a_{vr}$. Since $z_r(S) \leq z_r(T)$ and h_r is concave, the increment
 384 $h_r(z + a_{ur}) - h_r(z)$ is nonincreasing in z for every $a_{ur} \geq 0$. Multiplying by $w_r \geq 0$ and summing
 385 over r gives $\Delta(u \mid S) \geq \Delta(u \mid T)$. $\square$

386 *Proof of Theorem 4.* Let $B = 2$ and choose $\delta = \min\{\eta/2, 1/4\}$. One experience has candidates a, b
 387 with $c_a = \delta$, $F(\{a\}) = 2\delta$, $c_b = 1$, and $F(\{b\}) = 1$. Candidate a has density 2, candidate b has
 388 density 1, so max-density chooses a while OPT chooses b . The ratio is $2\delta \leq \eta$. $\square$

389 *Proof of Theorem 5.* Fix such a threshold. If the final cost $C(X) \geq B/2$, every accepted element
 390 had marginal density at least λ , so $F(X) \geq \lambda C(X) \geq (1 - \varepsilon)\text{OPT}/4$. Otherwise $C(X) < B/2$.
 391 Let O be optimal. By submodularity, $\text{OPT} \leq F(X) + \sum_{u \in O \setminus X} \Delta(u \mid X)$. Any missed optimal
 392 element was budget-feasible at its arrival. If it was below threshold, its marginal charge is at most λc_u ,
 393 and the total below-threshold charge is at most $\lambda B \leq \text{OPT}/2$. If it was threshold-admissible, the
 394 algorithm selected another candidate from the same group with at least as large raw marginal; these
 395 charges inject into accepted arrivals and sum to at most $F(X)$. Hence $\text{OPT} \leq 2F(X) + \text{OPT}/2$, so
 396 $F(X) \geq \text{OPT}/4$. $\square$

397 B Additional Experimental Details

398 This appendix records diagnostics omitted from the main story to keep the core paper within the
 399 9-page main-body budget. These results are useful for reproducibility and interpretation, but the main
 400 claims rely on exact package optima and denominator-matched exported-store scoring.

401 B.1 Controlled Package Generator

402 MEMAUDIT-Small samples a compact hidden event graph, exposes an ordered stream of experiences,
 403 and generates finite representation choices for each experience. Candidate types include raw span,
 404 atomic fact, entity summary, tombstone, and compound update. Costs are measured in normalized
 405 word-equivalent units. Evidence units correspond to future-query support requirements, including
 406 positive units for validity state: a tombstone is useful when a future query requires knowing that an
 407 older fact has been invalidated, superseded, deleted, or should trigger abstention.

408 The benchmark reports ratios only when the denominator is certified. For the 500-seed exact-small
 409 run, each budgeted package is solved exactly with the pure-Python branch-and-bound solver. A
 410 separate 1,200-row audit verifies equality with the MILP formulation for the clipped-coverage
 411 objective. The no-tombstone OPT ablation removes tombstone and compound-update candidates
 412 before solving the same exact problem; it is therefore a representation-frontier comparison, not an
 413 algorithmic failure mode.

414 B.2 Reader Failure Audit

415 The reader failure audit decomposes the LongMemEval-S focus diagnostic after the memory stores
 416 have been written. It is not a new denominator: it shows how often downstream errors come
 417 from missing retrieved evidence, abstention despite support, reader errors, unsupported answers, or
 418 scoring/parse uncertainty. This keeps the main package-ratio claim separate from retrieval and reader
 419 behavior.

420 B.3 Natural Package Construction

421 The Natural-200 package is built from support slices rather than full conversation histories. For each
 422 example, the candidate-generation stage receives source sessions and emits candidate memories with
 423 estimated costs. A separate annotation stage maps evidence units to future-query requirements and
 424 scores candidate coverage. We intentionally separate these stages so the writer does not see the query
 425 label during memory generation.

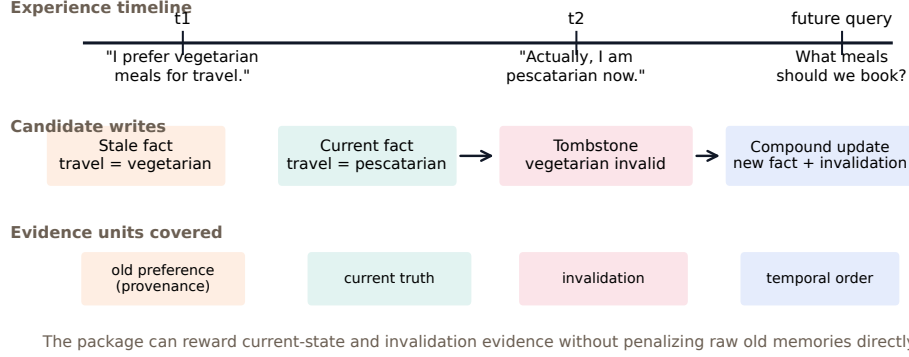


Figure 5: Illustrative validity-state package. A stale fact, current fact, tombstone, and compound update are distinct candidate writes with different costs and evidence coverage. The package optimum may keep at most one candidate from an experience group, so representation choice is non-trivial rather than a union of all useful strings. The objective rewards current truth, supersession, deletion, and abstention evidence so stale-fact avoidance can be scored at write time.

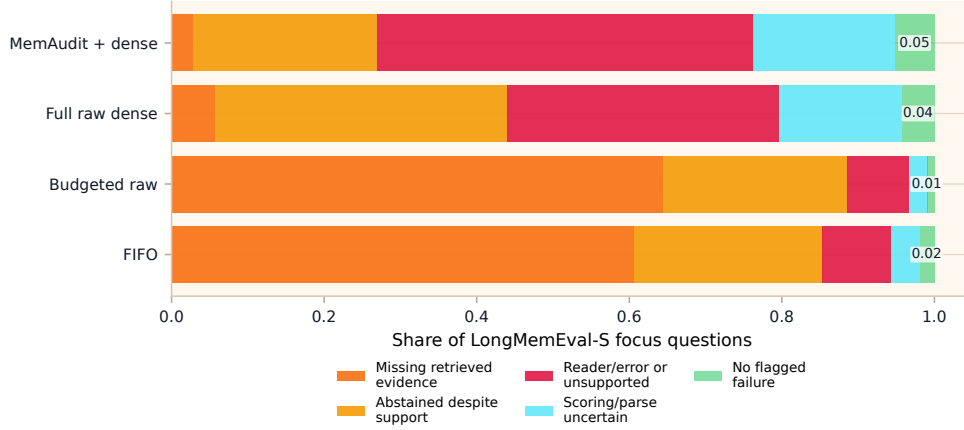


Figure 6: LongMemEval-S focus reader failure audit. Bars decompose retrieval/reader outcomes after write-time memory selection; the green segment marks examples with no flagged failure in this audit.

426 The annotation results should be read as model-adjudicated dataset construction with explicit audit
 427 hooks. The secondary 50-example audit surfaced ambiguous required-unit cases, which is why
 428 the main paper reports the stricter 87-example adjudicated subset for system diagnostics. For the
 429 released human coverage audit, two project annotators familiar with the schema independently label
 430 all model-positive coverage cells on the 87-example subset plus a stratified 20% sample of model-zero
 431 cells. We compute unweighted Cohen's κ on the full stratified audit frame, rather than reweighting to
 432 the package's natural positive/negative balance. Human-human disagreements are concentrated in
 433 sampled model-zero cells (116/162) and in summary/graph-edge/atomic-fact boundaries; human-
 434 Gemini disagreements show the same pattern (81/119 from sampled zero cells), indicating that the
 435 main ambiguity is borderline entailment rather than schema failure. The human-edited seed set
 436 validates schema usability and remains in the artifact as prompt/schema development data.

437 A Claude Sonnet 4.5 cross-model audit gives a useful boundary on this result. Re-adjudicating
 438 the package with Sonnet preserves package-writer rankings exactly at $B = 30, 60, 100$, but pro-
 439 duces lower cell-level agreement than Gemini-human (Table 3); we therefore treat Sonnet as a
 440 prompt/model-sensitivity stress test rather than an independent replacement annotator.

B.4 External System Exports

Mem0, A-Mem, and Letta are scored through exported written memories under a finite union denominator. The system memories are not forced into the package candidate schema; instead, they are added as new union candidates with their own costs and adjudicated coverage rows. This permits fair scoring of system-specific writes while preserving the exact denominator.

The main-system diagnostic reports post-hoc budget pruning because the public systems are not natively configured as package-budgeted writers. Recency pruning is a deployable baseline, salience pruning is a stronger heuristic, and upper pruning is an analysis-only bound over the system’s own exported memories. A high upper-pruned bound with a low salience-pruned score indicates that extraction produced useful memories but budget-aware selection failed.

C Cost and Constraint Sensitivity

We audit two package-design choices on the 87-example adjudicated natural subset. First, we relax the partition constraint from one candidate per experience to two candidates per experience. Second, we replace the default word-equivalent cost with a per-record-overhead rule $c(u) = 8 + \lceil \text{bytes}(u)/24 \rceil$. All rows below are deterministic recomputations from cached package/export artifacts and make no API calls.

Table 4: Package sensitivity on Natural-87. The $k = 2$ relaxation changes exact OPT modestly, and the byte-overhead cost increases absolute OPT because short atomic memories remain cheap relative to the tested budgets.

B	OPT $k = 1$	OPT $k = 2$	$k = 2/k = 1$	byte-overhead OPT	byte/default
30	0.931	0.943	1.013	1.310	1.322
60	1.356	1.374	1.027	1.483	1.107
100	1.489	1.540	1.060	1.632	1.131

Table 5 recomputes exported-system pruning at $B = 100$ against the package-candidate denominator under both cost rules. The exact union-denominator results in the main text remain the primary system diagnostic; this table isolates whether the small-budget ranking is mostly a word-count artifact. The qualitative ordering is stable: Letta salience remains strongest among non-oracle exported selections, Mem0 salience trails, and A-Mem full-store remains infeasible at this budget. A-Mem’s full notes are too large under both accounting rules: the mean minimum full-note cost is 1767.9 words and 463.5 byte-overhead units, far above $B = 100$.

Table 5: Exported-system cost sensitivity at $B = 100$, reported as selected value divided by package-candidate OPT under each cost rule. Upper rows use hidden coverage and are analysis-only.

Cost rule	Method	Ratio to package OPT
word	Letta salience	0.899
word	Mem0 salience	0.489
word	A-Mem metadata recency	0.190
word	A-Mem full native	0.000
byte-overhead	Letta salience	0.800
byte-overhead	Mem0 salience	0.442
byte-overhead	A-Mem metadata recency	0.266
byte-overhead	A-Mem full native	0.000

D LongMemEval Transfer Diagnostics

We also ran a LongMemEval-S retrieval/reader diagnostic. These runs are not oracle ratios because the full benchmark does not provide a finite candidate package with exact coverage rows. They are

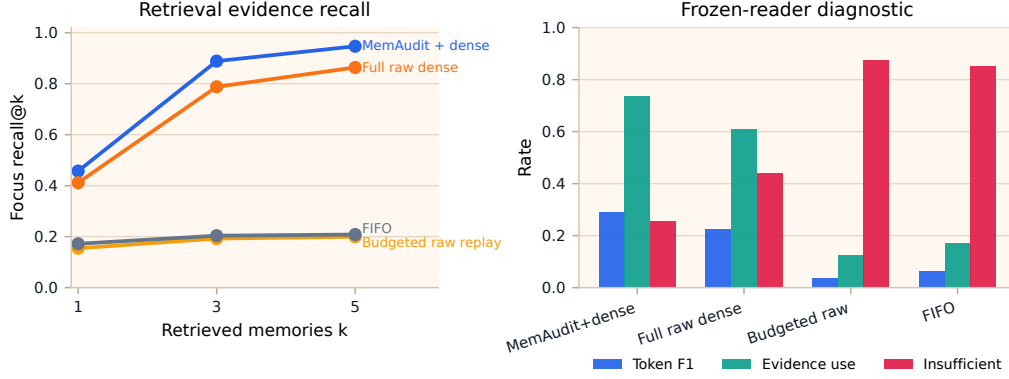


Figure 7: LongMemEval-S transfer diagnostics. These are evidence-use and reader diagnostics, not exact oracle-ratio results. Exact-match gains are not the basis of the main claim.

467 included to check whether package-style writing can reduce retrieval cost and improve evidence use
 468 in a more natural long-memory task.

469 The useful signal is narrower than end-to-end QA superiority. The MEMAUDIT-style store im-
 470 proved retrieval compactness and token-F1/evidence-use behavior relative to full raw-store dense
 471 retrieval, while exact-match changes were not the primary evidence. This is why the main paper uses
 472 LongMemEval only as a transfer diagnostic and does not claim significant answer-accuracy gains.

473 E Reproducibility Notes

474 The artifact contains deterministic exact-package runs, natural package exports, model-adjudication
 475 reports, and public-system scoring scripts. The safest reproduction path is to run the exact-small suite
 476 against certified OPT, audit a random subset with the MILP solver, re-score the 87-example
 477 adjudicated natural subset under package and union denominators, and recompute the public-system
 478 pruning diagnostics from exported Mem0, A-Mem, and Letta stores. API-backed natural package
 479 construction caches prompts, completions, token counts, model versions, and adjudication decisions.
 480 Natural package candidate generation used OpenRouter with
 481 google/gemini-3.1-flash-lite-preview, temperature 0, JSON response format, and a
 482 default 1400-token completion cap. Exported-system coverage/salience adjudication used cached
 483 Gemini Flash/Flash-Lite calls with temperature 0 and JSON response format; the Letta coverage and
 484 salience caps were 1800 and 4000 tokens respectively, while the Mem0 writer configuration used
 485 temperature 0 and a 700-token cap. The cross-model stress audit used OpenRouter
 486 anthropic/claude-sonnet-4.5 with temperature 0, JSON response format, cached
 487 package-level adjudications, and a separate cached 1,975-cell binary coverage pass. All cached calls
 488 include prompt hashes and token/cost metadata. The exact oracle denominator itself is deterministic
 489 once a package has been constructed.

NeurIPS Paper Checklist

1. **Claims.** [Yes] The abstract and introduction state that MEMAUDIT is an exact-oracle evaluation protocol for finite memory-writing packages, not a deployed memory architecture. The scope and package-conditional nature are discussed in [Section 8](#).
2. **Limitations.** [Yes] [Section 8](#) gives a separate scope and limitations section covering package-conditional optima, model-adjudicated natural labels, support-sliced natural packages, exact-solver scale, and downstream QA diagnostics.
3. **Theory, assumptions, and proofs.** [Yes] Assumptions for the semantic coverage theorem and scoped oracle-GVT guarantee are stated in [Sections 2](#) and [3](#); complete proofs are provided in [Section A](#).
4. **Experimental result reproducibility.** [Yes] The artifact includes deterministic package generators, exact solvers, cached package artifacts, result summaries, and rerun commands. Minimal reproduction does not require API calls; API-backed reconstruction is documented separately.
5. **Open access to data and code.** [Yes] The anonymized submission includes code, package data, Croissant metadata, and documentation for reproducing the main exact-package and exported-system diagnostics.
6. **Experimental setting/details.** [Yes] [Sections B](#) and [4](#) to [6](#) describe the package distributions, budgets, candidate families, adjudication path, exact solvers, and exported-system scoring setup.
7. **Experiment statistical significance.** [Yes] The controlled exact sweep reports bootstrap 95% confidence intervals over the canonical 500-seed run. Natural and exported-system package rows are reported as deterministic diagnostics on the adjudicated subset.
8. **Experiments compute resources.** [Yes] [Section 8](#) and the artifact documentation state that exact-small and cached natural re-scoring run on CPU without API calls. API-backed natural annotation and external memory exports require OpenRouter-compatible API access and cached model/version metadata.
9. **Code of ethics.** [Yes] The work is an evaluation artifact for memory-writing systems. The release is anonymized for review and excludes API keys, private review notes, and local environment files.
10. **Broader impacts.** [Yes] The main risk is that memory benchmarks can incentivize storing sensitive or stale personal information. [Sections 5](#) and [8](#) discuss validity-state memories, abstention/deletion units, and auditability as mitigation-oriented design choices.
11. **Safeguards.** [N/A] The paper does not release a pretrained model or high-risk generative model. The artifact releases package schemas, generators, scoring code, and cached benchmark data.
12. **Licenses.** [Yes] The paper cites external systems and datasets used for comparison. The artifact documentation includes dependency and asset manifests; external repository checkouts are not bundled in the anonymized artifact.
13. **Assets.** [Yes] The submission includes a dataset/evaluation artifact with README, reproducibility instructions, evaluation card, artifact manifest, Croissant metadata, and documented intended/not-intended uses.
14. **Crowdsourcing and research with human subjects.** [N/A] No paid crowd workers or human-subject experiments are used. The human-edited seed package consists of fictional examples edited for schema validation and does not include participant data.
15. **IRB approvals.** [N/A] The released examples are synthetic, model-adjudicated support slices, or fictional human-edited seed examples. No identifiable participant data or human-subject intervention is released.
16. **Declaration of LLM usage.** [Yes] LLMs are used for natural support-slice package construction, adjudication, external memory-system exports, and reader diagnostics; these uses are described in [Sections B](#), [D](#), [6](#) and [8](#).